Bucket Aggregation in Bucket-Based Priority Queues for Heuristic Search

Anonymous submission

Abstract

Bucket-based priority queues can reduce the overhead of priority queue operations for A* and related bounded-suboptimal and anytime variants, especially when transition costs are small integers. The buckets are usually indexed by both f-cost (primary) and h-cost (secondary), enabling efficient tie-breaking and priority ordering. However, when the optimal solution cost is large and the heuristic is informative, the number of secondary buckets within a primary bucket can become very large, leading to sparsely populated buckets that degrade performance. To address this, we introduce a simple aggregation method that restores bucket density while preserving standard solution-quality guarantees. We demonstrate its effectiveness in several benchmark domains.

Introduction

For search problems with a small number of integer transition costs, it is well-known that the performance of both Dijkstra's algorithm and A* can be improved by using a *bucket queue* data structure for the priority queue, instead of a binary heap (Dial 1969; Burns et al. 2012). Recently, a *bucket heap* data structure was proposed that generalizes the bucket queue approach to bounded-suboptimal and anytime variants of A* that expand nodes in order of a non-admissible node evaluation function (Fereday and Hansen 2025). Both bucket queues and bucket heaps group nodes with the same priority (i.e., equal evaluation function value) into buckets from which they can be expanded in any order, reducing the overhead of priority queue operations compared to binary heaps and often supporting near-constant-time operations in favorable cases, though without a general guarantee.

In this paper, we address a complication that can affect the performance of both bucket queues and bucket heaps when buckets are indexed by both the f-cost and h-cost of nodes. In this two-tiered approach to bucketing, a *primary bucket* is the set of nodes with the same f-cost, and a *secondary bucket* is the subset of nodes within a primary bucket that share the same h-cost. For A*, this two-tier indexing enables efficient tie-breaking on h-cost among nodes with identical f-cost. For bounded-suboptimal and anytime variants of A*, indexing buckets in this way is required when the evaluation function weights the g-cost and h-cost differently to determine node priority. Thus in both cases, two-tiered indexing supports more efficient ordering of node expansions.

When A* solves search problems with integer edge costs bounded by a small constant, the f-costs of nodes in the priority queue are typically in a small range. Even so, when the heuristic h is informative, the number of distinct h-costs within a single f-level can grow proportionally to the optimal solution cost. Thus, when the optimal f-cost is very large, an informative heuristic can counterintuitively increase priority queue overhead by generating many distinct h-costs within a single f-level, with the resulting bucket sparsity negating the constant-time benefits of the bucket structure. This issue is further exacerbated when the range of f-costs in the priority queue also increases due to increased transition costs or the use of a non-admissible heuristic by bounded-suboptimal and anytime variants of A*.

To limit secondary bucket sparsity, we propose a simple approach to aggregating secondary buckets. By trading a small loss of resolution in h-based ordering for improved bucket density, we reduce overhead without loss of solution quality guarantees. For integer-cost search domains with high optimal f-cost, our experiments show that bucket aggregation improves the performance of bucket-based priority queues when bucket density would otherwise be sparse.

Background

A* and its bounded-suboptimal and anytime variants use a priority queue to organize the search, ordered by a function of both $g(n)$, the cost of the best path found so far from the start node to node n, and $h(n)$, a lower bound on the cost of a path from n to a goal node. Different search algorithms are primarily distinguished by their node evaluation function.

The priority queue has three basic operations: *enqueue*, which inserts a node; *dequeue*, which removes the next node in order of priority; and *changePriority*, which reorders a node n in the queue when a shorter path to n is found that decreases $g(n)$. For bounded-suboptimal and anytime variants of A* that change the priority function during the search, an additional *reorder* operation rebuilds the priority queue.

When a priority queue is implemented as a binary heap, *enqueue*, *dequeue* and *changePriority* each take worst-case logarithmic time in the size of the queue. The average-case complexity of *dequeue* and *changePriority* is also logarithmic in the size of the queue, while the average-case complexity of *enqueue* is constant. For a binary heap, *reorder* takes worst- and average-case linear time.

Two-level bucket queue

To support tie-breaking among nodes with equal f-cost in favor of nodes with smaller h-cost, a bucket queue for A* is typically indexed by both f-cost and h-cost (Burns et al. 2012). The next node to be expanded is identified by first selecting the non-empty primary bucket with least f-cost, and then selecting the non-empty secondary bucket within it with least h-cost.

In this data structure, an $f_{\min}$ index keeps track of the least f-cost non-empty primary bucket. Because it increases monotonically, it can be updated in amortized constant time. For each primary bucket, a similar $h_{\min}$ index keeps track of the least h-cost non-empty secondary bucket. Unlike $f_{\min}$, however, $h_{\min}$ is not monotonic: a successor node can have the same f-cost but a smaller h-cost (since increases in g may be offset by decreases in h), causing $h_{\min}$ to decrease.

If expanding a node from the secondary bucket indexed by $h_{\min}$ generates a successor node with the same f-cost but smaller h-cost, $h_{\min}$ can be updated in constant time. However, when the last node in a secondary bucket is expanded, locating the next non-empty secondary bucket may require a worst-case linear scan over secondary buckets. The complexity of locating the next non-empty secondary bucket can be reduced to worst-case logarithmic time by maintaining a binary heap over non-empty secondary buckets. But in practice, this does not typically improve empirical performance.

Bucket heap

Fereday and Hansen (2025) introduced a *bucket heap* priority queue data structure that generalizes the two-level bucket queue to support bounded-suboptimal and anytime variants of A*. A bucket heap maintains the same two-level bucket queue indexed by f-cost and h-cost, but augments it with a binary heap whose elements are the non-empty primary buckets. The heap orders the primary buckets by a dynamic priority that is determined by the node evaluation function, using h-cost equal to h_{min} for the primary bucket and g-cost equal to the f-cost associated with the primary bucket minus h_{min}. Thus the dynamic priority changes only when the active secondary bucket within the primary bucket (indexed by h_{min}) changes. The overhead for changing the dynamic priority is logarithmic in the number of non-empty primary buckets, and thus almost negligible in most cases. In practice, the overhead for maintaining a bucket heap is dominated by the overhead for maintaining the underlying bucket queue, which is asymptotically greater.

Because a bucket heap is built on top of a bucket queue, it has the advantage that it provides *two* priority queues: a bucket heap ordered by the non-admissible node evaluation function and an underlying bucket queue ordered by f-cost. This is especially useful because many bounded-suboptimal and anytime variants of A* require a second priority queue ordered by f-cost (e.g., Thayer and Ruml 2008; Richter and Westphal 2010; Thayer and Ruml 2011; Gilon, Felner, and Stern 2016; Aine et al. 2016; Cohen et al. 2018; Natarajan et al. 2019; Fereday and Hansen 2025). Those search algorithms that do not require a second priority queue ordered by f-cost can still take advantage of it to compute a tighter suboptimality bound than would otherwise be available.

Secondary bucket aggregation

After briefly describing some additional details about our implementation, we introduce our approach to aggregation.

Implementation details

Primary buckets are stored in a dynamic array indexed by f, with a pointer $f_{\min}$ to the smallest f-value of any non-empty bucket. The array can be restricted to the range $[f_{\min}, C]$, given an upper bound C on solution cost (e.g., obtained from a suboptimal solution). Each entry in the array corresponds to a primary bucket and stores a pointer to a data structure that organizes the nodes with that f-value.

Each primary bucket maintains a dynamic array of secondary buckets, indexed by h, together with a pointer $h_{\min}$ to the smallest h-value of any non-empty secondary bucket. Each secondary bucket is implemented as a dynamic array storing pointers to nodes (allocated from a pre-allocated node pool), supporting amortized constant-time insertion. Because the order of nodes within a secondary bucket is irrelevant, deletions can be performed in constant time. When removing a specific node (e.g., due to a decrease in g-cost), the last element is swapped into its position to avoid shifting elements; for dequeue operations, the last element is removed directly, which also improves cache locality.

Bucket aggregation and solution quality guarantees

When secondary buckets are indexed by h over a large range, this can lead to many empty or near-empty buckets. This is a key factor affecting performance: it increases memory usage and allocation overhead, reduces cache locality, and increases the cost of finding the next non-empty secondary bucket.

To mitigate these effects, we aggregate nearby secondary buckets into coarser buckets that cover ranges of h-values. For simplicity, we partition the h-value range into intervals of size α, where α is a tunable parameter. All nodes in the same aggregated bucket b share a representative value $\hat{h}(b)$, which determines the priority assigned to the bucket.

For A*, aggregation preserves optimality regardless of how $\hat{h}(b)$ is chosen, because all nodes within a primary bucket share the same f-cost, and aggregation affects only tie-breaking among nodes with equal f. Therefore, we simply assign each node n with h-value $h(n)$ to a secondary bucket indexed by $\hat{h}(b) = \lfloor h(n)/\alpha \rfloor$. For bounded-suboptimal and anytime variants of A*, preserving suboptimality bounds requires a more careful choice of the h-value $\hat{h}(b)$ associated with each aggregated bucket b.

For search algorithms that expand nodes in increasing order of a weighted evaluation $f_w(n) = g(n) + w \cdot h(n) = f(n) + (w-1)h(n)$, we set $\hat{h}(b)$ to the minimum possible h-value of any node in the aggregated bucket b. Let f denote the f-value associated with the primary bucket containing b. Since $\hat{h}(b) \leq h(n)$ for all $n \in b$, the resulting bucket priority

$$\hat{f}_w(b) = f + (w-1)\hat{h}(b)$$

is a lower bound on $f_w(n)$ for every node n in b. Thus, nodes are expanded in nondecreasing order of a lower bound on f_w, which preserves the standard suboptimality guarantee.

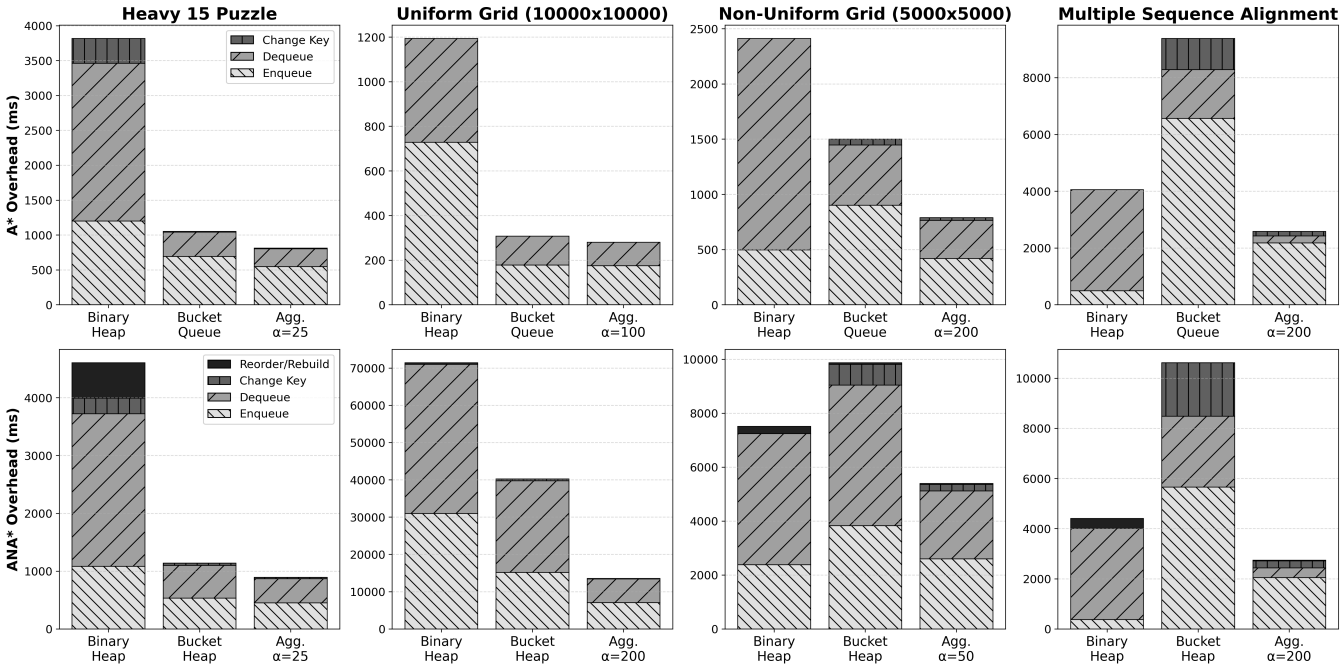

Figure 1: Priority queue overhead (in milliseconds) for A* (above) and ANA* (below) using a binary heap, a standard bucket queue ($\alpha = 1$), and a bucket queue/heap with secondary bucket aggregation based on the indicated aggregation factor α.

For algorithms such as ANA* (Stern et al. 2014) and Dynamic Potential Search (Gilon, Felner, and Stern 2016), which expand nodes in *decreasing* order of a potential function, the analysis is reversed because their priority functions are decreasing in $h(n)$. ANA* expands nodes in decreasing order of the potential $u(n) = (C - g(n))/h(n)$, where C is the cost of the best solution found so far. Using $g(n) + h(n) = f$, this can be written as $u(n) = 1+(C-f)/h(n)$, which is decreasing in $h(n)$ (since $C \geq f$ for any open node). Therefore, we set $\hat{h}(b)$ to the *maximum* possible h-value in bucket b, so that the resulting bucket priority is a lower bound on the potential of any node in the bucket. This preserves the suboptimality bound. Similar reasoning applies to Dynamic Potential Search.

More generally, aggregation preserves suboptimality guarantees whenever the induced bucket priority is a lower bound on the true priority function. Note that the resulting order of expansion is more conservative, in the sense that it may delay the expansion of nodes with higher true priority within a bucket, in order to preserve the suboptimality bound despite the coarsening of node priorities due to aggregation.

For bounded-suboptimal and anytime variants of A*, the effect of bucket aggregation on node expansion order is limited (though not eliminated), since pruning decisions are based on bounds that are unaffected by aggregation. A similar idea can be applied to A* to mitigate the loss of tie-breaking resolution when aggregating secondary buckets with different h-values. Once a goal node is generated, A* can terminate with this solution as soon as it begins expanding a primary bucket with the same f-value. This works because all nodes n in that bucket have $f(n) \geq g(\text{goal})$.

Experiments and analysis

We evaluated our approach to bucket aggregation in the following domains, all with integer transition costs, high optimal solution costs, and relatively informative heuristics.

- The heavy 15-puzzle, where tile movement cost equals the tile's label, using the heavy Manhattan distance heuristic and averaged over Korf's (1985) 100 instances. Optimal solution costs are in the range 400 to 450.
- Path planning in a $10{,}000 \times 10{,}000$ grid with cardinal moves, unit transition costs, and the Manhattan distance heuristic. Results are averaged over 25 random instances with the start state in the upper left corner, the goal state in the bottom right corner, and obstacles generated by blocking 20% of cells uniformly at random. Optimal solution costs are around 20,000 in all instances.
- Path planning in a similar $5{,}000 \times 5{,}000$ grid, also averaged over 25 random instances, but with uniform random integer transition costs from 1 through 5. Optimal solution costs average around 37,000.
- Alignment of six sets of five protein sequences each of length approximately 450 from the same dataset as Stern et al. (2014), but with PAM250 instead of unit costs (Lermen and Reinert 2000). The sum-of-pairs heuristic is used and optimal solution costs average around 64,000.

A* and ANA*. The top and bottom rows of Figure 1 compare priority queue overhead incurred by A* and ANA*, respectively, in solving these search problems using a binary heap, a two-level bucket queue without aggregation ($\alpha = 1$), and a bucket queue with secondary buckets aggregated based on an aggregation factor α tuned for each problem.

Domain	Queue	Time (s)	Page F.	LLC M.	Branch M.
Grid	Unaggregated	97.02	234K	109M	1.62B
	Aggregated	**47.22**	**8K**	**40M**	**0.49B**
MSA-5	Unaggregated	60.60	1.24M	1.13B	142M
	Aggregated	**49.90**	**55K**	**0.94B**	**90M**

Table 1: Hardware profiling of priority queue overhead in ANA* on non-uniform grid and MSA instances. Bucket aggregation reduces memory footprint (page faults, LLC misses) and improves CPU predictability (branch misses).

Priority Queue	ϵ	Speedup PQ	Speedup Overall
Binary Heap	1.04	1.00x	1.00x
Bucket Heap ($\alpha = 1$)	1.04	0.10x	0.64x
Bucket Heap ($\alpha = 2000$)	**1.04**	**2.21x**	**1.85x**

Table 2: Performance of Dynamic Potential Search on a 6-sequence MSA instance. Secondary bucket aggregation ($\alpha = 2000$) removes the priority queue (PQ) bottleneck, yielding a substantial overall speedup.

The search domains are ordered from left to right in increasing order of solution cost, showing that as the solution cost increases (and assuming an informative heuristic), both the benefit of secondary bucket aggregation and the optimal aggregation factor α tend to increase. Although the results are not shown in the figure, we also tested bucket aggregation in solving problems with smaller solution costs, such as the 15-puzzle with unit costs. For such problems, bucket aggregation can provide some limited improvement, but the best α can be 5 or less, and the improvement is slight.

The results show that for the heavy 15-puzzle and uniform grid, an unaggregated bucket queue already outperforms a binary heap, and aggregation improves performance further. In contrast, for Multiple Sequence Alignment (MSA) and the non-uniform grid — the two domains with the largest solution costs — a bucket queue without aggregation can perform even worse than a binary heap, particularly for ANA*. Because both domains have non-uniform transition costs spanning a wider range than the uniform grid, they maintain many active primary buckets at once, in addition to many secondary buckets for each primary bucket. The resulting bucket sparsity leads to increased overhead from memory allocation, scanning, and cache misses.

Figure 1 shows that secondary bucket aggregation restores bucket density and thus the efficiency of bucketing across all domains. This interpretation is supported by the results in Table 1, which presents hardware profiling of priority queue overhead for ANA* on non-uniform grid and MSA problem instances. The table shows that bucket aggregation not only reduces total solving time, but does so by substantially reducing page faults, last-level cache (LLC) misses, and branch misses. The large reduction in page faults demonstrates a substantial reduction in dynamic memory allocation overhead. The drop in LLC misses indicates that aggregation improves cache locality. Fewer branch misses reflect not having to scan as frequently for the next non-empty secondary bucket, as well as not having to scan as far.

In all domains, the improved performance of bucketing stems primarily from faster *dequeue* operations. Some of this gain comes from better cache locality, but most comes from reduced scanning costs when searching for the next non-empty secondary bucket, as well as fewer primary heap operations as the result of less switching between secondary buckets. The efficiency of the *enqueue* and *changePriority* operations is sensitive to cache locality and so it is also substantially improved by secondary bucket aggregation. The overhead from cache misses could potentially be reduced further by aggregating primary buckets as well, especially in the non-uniform grid and MSA domains, where non-uniform transition costs widen the range of f-values in the priority queue. We revisit this possibility in the conclusion.

Dynamic Potential Search. Finally, Table 2 shows the performance of the bounded-suboptimal algorithm Dynamic Potential Search (DPS) (Gilon, Felner, and Stern 2016) on a more difficult 6-sequence MSA instance, comparing the three priority queue implementations at a suboptimality bound of $\epsilon = 1.04$. The sequences are from the same dataset used in the results for 5-sequence alignment, and the solution found has cost 96,206.

For this problem, without aggregation, the bucket heap performs far worse than a binary heap, achieving only $0.10\times$ the priority queue throughput and reducing overall search speed to $0.64\times$ that of the binary heap baseline. With secondary bucket aggregation at $\alpha = 2000$, however, the trend reverses. The aggregated bucket heap not only outperforms the standard bucket heap, it achieves $2.21\times$ the priority queue throughput of the binary heap, translating to an overall speedup of $1.85\times$. This result illustrates how bucket aggregation can extend the range of search problems for which bucket-based priority queues are useful.

Conclusion

Our results confirm the effectiveness of bucketing in reducing priority overhead when solving challenging search problems, but also reveal its limitations for problems with high solution cost (and an informative heuristic) and how these limitations can be overcome. When used to offset these challenges, secondary bucket aggregation trades a minor loss of tie-breaking and priority resolution for a restoration of memory and cache density, leading to improved scalability for bucket-based search.

Interestingly, secondary bucket aggregation creates an intermediate data structure between a standard two-level bucket queue for A* and Dial's one-level bucket queue, since the latter essentially aggregates all secondary buckets into a single bucket. From this perspective, our approach provides a principled tradeoff between these two designs.

An obvious direction for future work is to consider aggregating primary buckets as well, which we noted above could further reduce overhead in domains with non-uniform transition costs such as the non-uniform grid and MSA. Primary bucket aggregation is also necessary in order to extend bucketing to search problems with real-valued transition costs.

References

Aine, S.; Swaminathan, S.; Narayanan, V.; Hwang, V.; and Likhachev, M. 2016. Multi-Heuristic A*. *The International Journal of Robotics Research*, 35(1-3): 224–243.

Burns, E.; Hatem, M.; Leighton, M.; and Ruml, W. 2012. Implementing Fast Heuristic Search Code. *Proc. of the 5th Annual Symposium on Combinatorial Search*, 25–32.

Cohen, L.; Greco, M.; Ma, H.; Hernandez, C.; Felner, A.; Kumar, T.; and Koenig, S. 2018. Anytime focal search with applications. In *Proc. of the 27th International Joint Conference on Artificial Intelligence*, 1434–1441.

Dial, R. 1969. Shortest-path forest with topological ordering. *Communications of the ACM*, 12(11): 632–633.

Fereday, G.; and Hansen, E. A. 2025. A Bucket-Based Priority Queue for Bounded-Suboptimal and Anytime A* Search. In *Proc. of the 18th International Symposium on Combinatorial Search*.

Gilon, D.; Felner, A.; and Stern, R. 2016. Dynamic potential search – a new bounded suboptimal search algorithm. In *Proc. of the 9th Annual Symposium on Combinatorial Search*, 36–44. AAAI press.

Korf, R. 1985. Depth-first iterative deepening: An optimal admissible tree search. *Artificial Intelligence*, 27(1): 97–109.

Lermen, M.; and Reinert, K. 2000. The Practical Use of the A* Algorithm for Exact Multiple Sequence Alignment. *Journal of Computational Biology*, 7: 655 – 671.

Natarajan, R.; Saleem, M.; Aine, S.; Likhachev, M.; and Choset, H. 2019. A-MHA*: Anytime Multi-Heuristic A*. *Proceedings of the 12th Annual Symposium on Combinatorial Search*, 10(1): 192–193.

Richter, S.; and Westphal, M. 2010. The LAMA Planner: Guiding Cost-Based Anytime Planning with Landmarks. *Journal of Artificial Intelligence Research*, 39: 127–177.

Stern, R.; Felner, A.; van den Berg, J.; Puzis, R.; Shah, R.; and Goldberg, K. 2014. Potential-based bounded-cost search and Anytime Non-Parametric A*. *Artificial Intelligence*, 214: 1–25.

Thayer, J.; and Ruml, W. 2008. Faster than Weighted A*: An Optimistic Approach to Bounded Suboptimal Search. In *Proc. of the 18th International Conference on Automated Planning and Scheduling*, 355–362.

Thayer, J.; and Ruml, W. 2011. Bounded suboptimal search: A direct approach using inadmissible estimates. In *Proc. of the 22nd International Joint Conference on Artificial Intelligence*, 674–679.